

İÇİNDEKİLER

İÇİNDEKİLER.....	1
GİRİŞ.....	1
SİSTEM MİMARİSİ	1
OYUN MEKANİKLERİ VE TASARIMI	2
Nişan Alma Mekanığı (Cinemachine Priority)	3
Ateş Etme ve Geri Tepme Mekanığı (Raycast).....	3
Şarjör Değiştirme Sistemi (Coroutine)	4
Oyuncu Can ve Hasar Alma Sistemi.....	5
Yapay Zeka Durum Makinesi (State Machine)	6
KULLANILAN YAZILIMSAL MİMARİLER VE TEKNİKLER	7
Nesne Yönelimli Programlama (OOP) Prensipleri	7
Kapsülleme (Encapsulation) - PlayerHealth Örneği.....	8
Sorumlulukların Ayrılması - Script'lerin (AteşEtme, PlayerHealth) ayrı olması)	8
Tasarım Desenleri: Durum Makinesi (State Machine)	8
Eşzamansız Programlama (Asynchronous): Coroutine'ler.....	8
Unity'e Özgü Teknikler	9
Cinemachine Kamera Yönetimi:	9
Animation Rigging (IK):.....	9
NavMesh Navigasyonu:	9
TASARLANAN ARAYÜZLER (UI/UX)	10
Oyun İçi Arayüz (HUD)	10
Oyun Sonu Paneli (Game Over Panel)	10
GRAFİK KULLANIMI VE OPTİMİZASYON	11
Kullanılan Grafik Varlıkları (Assets) ve Kaplamalar	11
Görsel Efektler (VFX) ve Işıklandırma	11
Optimizasyon Teknikleri.....	12
SÜREÇ YÖNETİMİ VE GITHUB KULLANIMI	12
Ekip Görev Dağılımı	12
GitHub Versiyon Kontrol Süreci	12

LİTERATÜR TARAMASI VE KARŞILAŞTIRMA	12
Örnek Çalışmaların İncelenmesi	13
Projemiz ile Literatürün Karşılaştırılması	13
KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER.....	13
Teknik Zorluklar	14
Süreç Yönetimi Zorlukları.....	14
Zorluk (GitHub Merge Çakışması):.....	14
SONUÇ VE PROJENİN KATKILARI.....	15
KAYNAKÇA	15

GİRİŞ

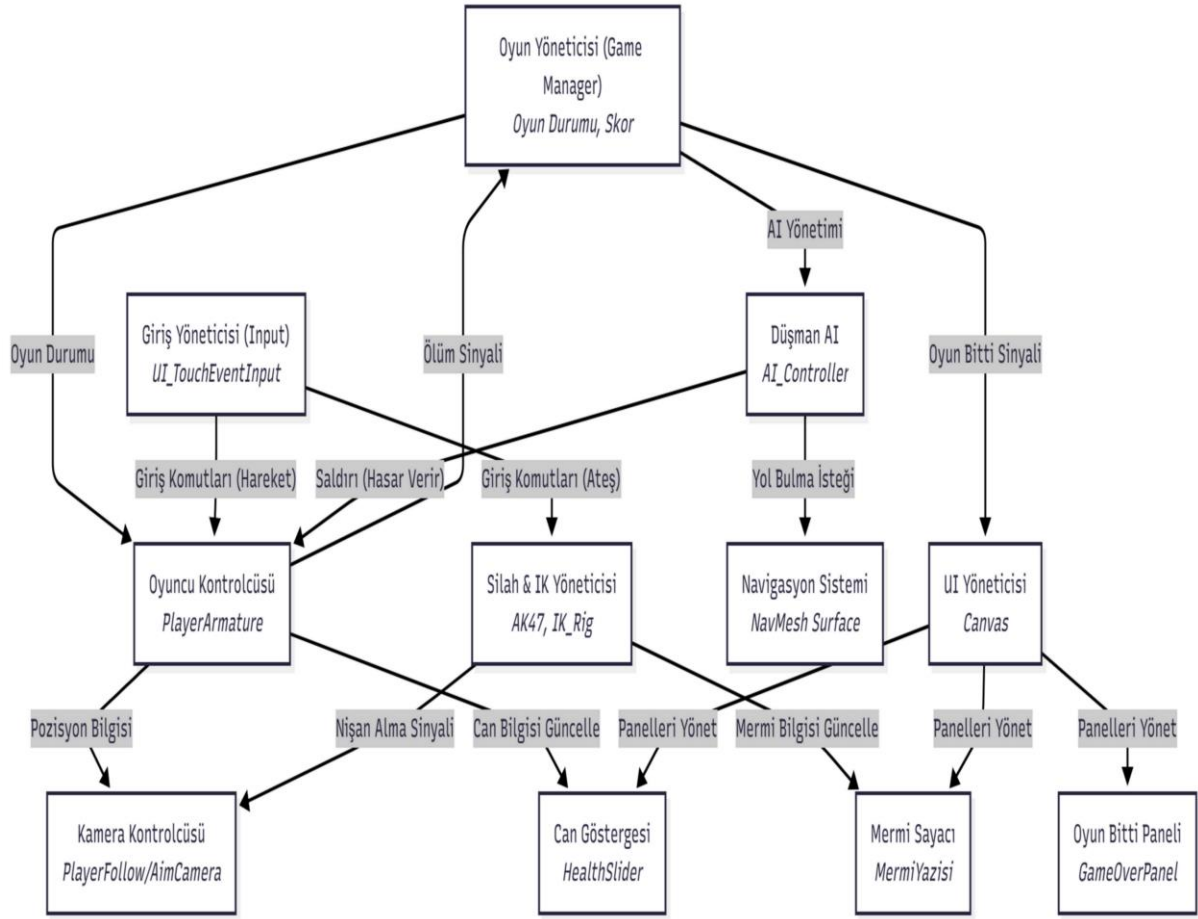
Bu rapor, Yazılım Geliştirme Laboratuvarı dersi kapsamında, TPS-AI Projesi adlı projenin geliştirme sürecini ve teknik detaylarını sunmak amacıyla hazırlanmıştır. Projenin temel amacı, Unity oyun motoru kullanılarak, modern ve fonksiyonel bir üçüncü şahıs nişancı (Third Person Shooter - TPS) oyunu oluşturmaktır.

Bu amaç doğrultusunda projemiz nişan alma, ateş etme, can sistemi gibi temel oyuncu mekaniklerini ve bu mekaniklere tepki verebilen bir düşman yapay zekasını içermektedir. Proje geliştirme sürecinde, C# programlama dili ve Nesne Yönelimli Programlama (OOP) prensipleri temel alınmıştır.

Projenin teknik altyapısında, Unity'nin 'Cinemachine' paketi (kamera yönetimi için), 'NavMesh' sistemi (yapay zeka navigasyonu için), 'Animation Rigging (IK)' paketi (gerçekçi silah tutma ve nişan alma için) ve 'Durum Makinesi' (State Machine) tasarım deseni (yapay zeka davranışları için) gibi modern teknolojiler ve yöntemler kullanılmıştır.

Proje, bir ekip çalışması olarak yürütülmüş ve tüm geliştirme süreci 'GitHub' versiyon kontrol sistemi üzerinden yönetilmiştir. Bu rapor, projenin sistem mimarisinden başlayarak , geliştirilen tüm oyun mekaniklerini, kullanılan yazılımsal mimari ve teknikleri, süreç yönetimi ve literatür karşılaştırmasını adım adım detaylandıracaktır.

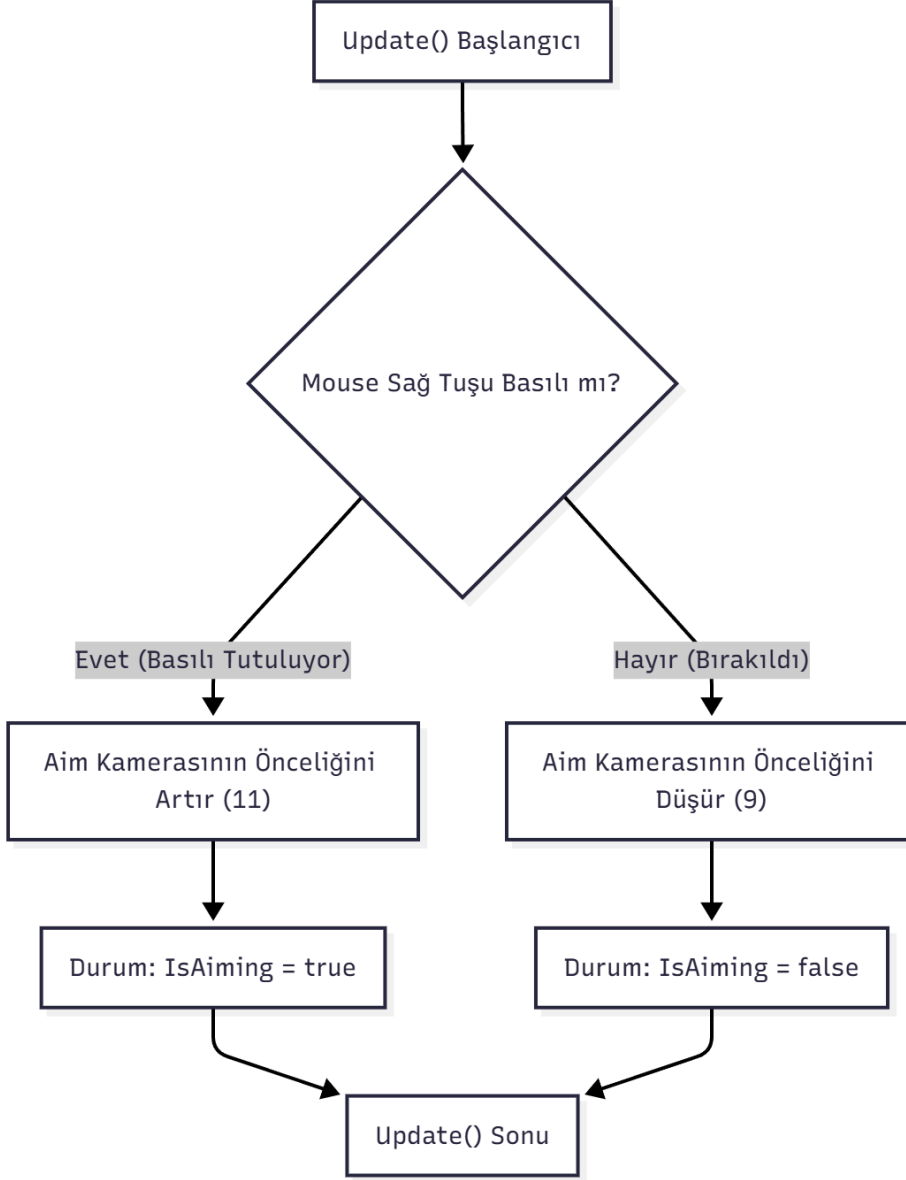
SİSTEM MİMARİSİ



Projenin sistem mimarisi birbiriyle iletişim halinde olan modüler bileşenlerden oluşmaktadır . Mimarinin merkezinde, oyunun genel durumunu (oyunun bitmesi vb.) yöneten bir 'Oyun Yöneticisi' bulunmaktadır. 'Oyuncu Bileşenleri' (Input, Player Controller, Silah Yöneticisi) oyunculardan gelen girdileri işlerken, 'Arayüz (UI) Bileşenleri' bu verileri (can, mermi sayısı) görsel olarak kullanıcıya sunar. Bu modüler yapı her sistemin kendi sorumluluğuna odaklanmasını sağlayarak kodun okunabilirliğini ve yönetimini kolaylaştırmıştır.

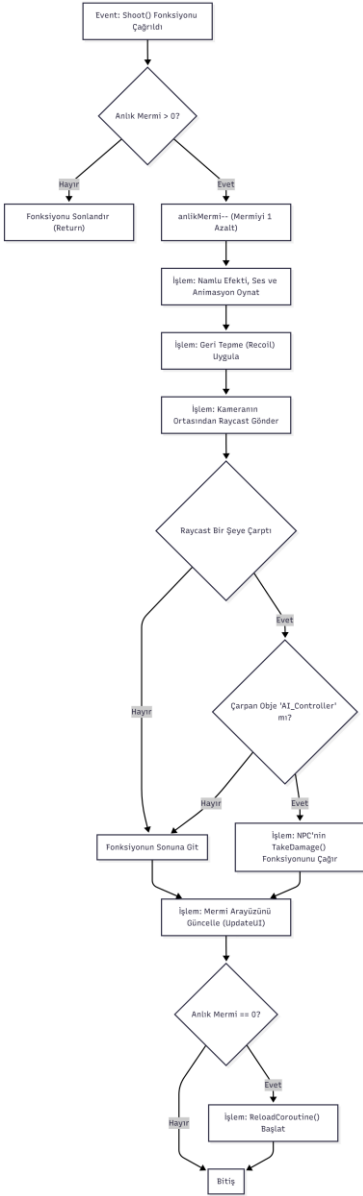
OYUN MEKANİKLERİ VE TASARIMI

Nişan Alma Mekaniği (Cinemachine Priority)



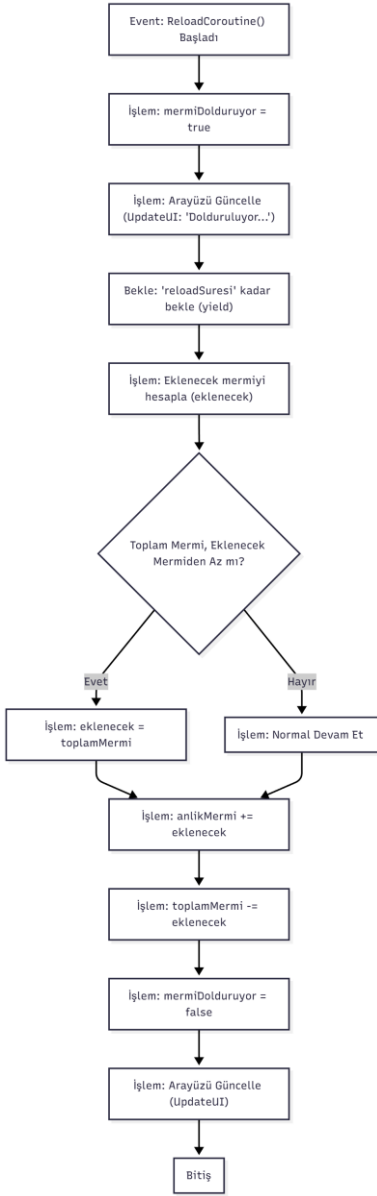
Projemizdeki nişan alma mekaniği NisanAlmaSistemi.cs script'i tarafından yönetilmektedir. Bu sistem, oyuncunun 'Mouse Sağ Tuşu'na basılı tutup tutmadığını her karede denetler. Tuşa basılı tutulduğu sürece Cinemachine 'Aim Camera' (Nişan Kamerası) objesinin 'Priority' (Öncelik) değeri 11'e yükseltilir. Bu Cinemachine'in otomatik olarak normal kameradan nişan kamerasına geçiş yapmasını sağlar. Tuş bırakıldığında ise 'Priority' değeri 9'a düşürülerek tekrar normal kamera görünümüne dönlür. Bu öncelik (priority) tabanlı yaklaşım, kod karmaşıklığı olmadan akıcı bir kamera geçişi elde etmemize sağlamıştır.

Ateş Etme ve Geri Tepme Mekaniği (Raycast)



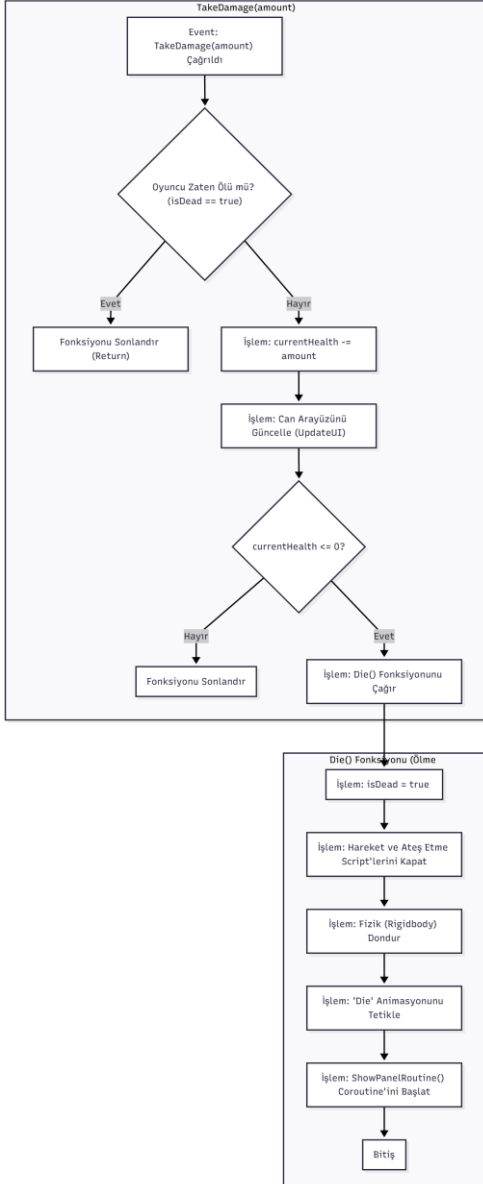
Ateş etme süreci, AtesEtme.cs script'i içindeki Shoot() fonksiyonu ile tetiklenir. Sistem öncelikle anlık mermi sayısının sıfırdan büyük olup olmadığını kontrol eder. Mermi varsa mermi sayısı bir azaltılır, 'Namlu Ateşi Efekti' (VFX), 'Ateş Sesi' (SFX) ve 'Shoot' animasyonu eşzamanlı olarak tetiklenir. Merminin hedefini bulması için kameranın ekran merkezinden ileriye doğru bir Raycast (Işın) gönderilir. Bu ışın, 'AI_Controller' script'ine sahip bir hedefe isabet ederse, hedefin TakeDamage() fonksiyonu çağrılarak hasar verilir. Her atış sonrası UpdateUI() ile arayüz güncellenir ve şarjör boşalırsa otomatik olarak 'Reload' süreci başlatılır.

Şarjör Değişirme Sistemi (Coroutine)



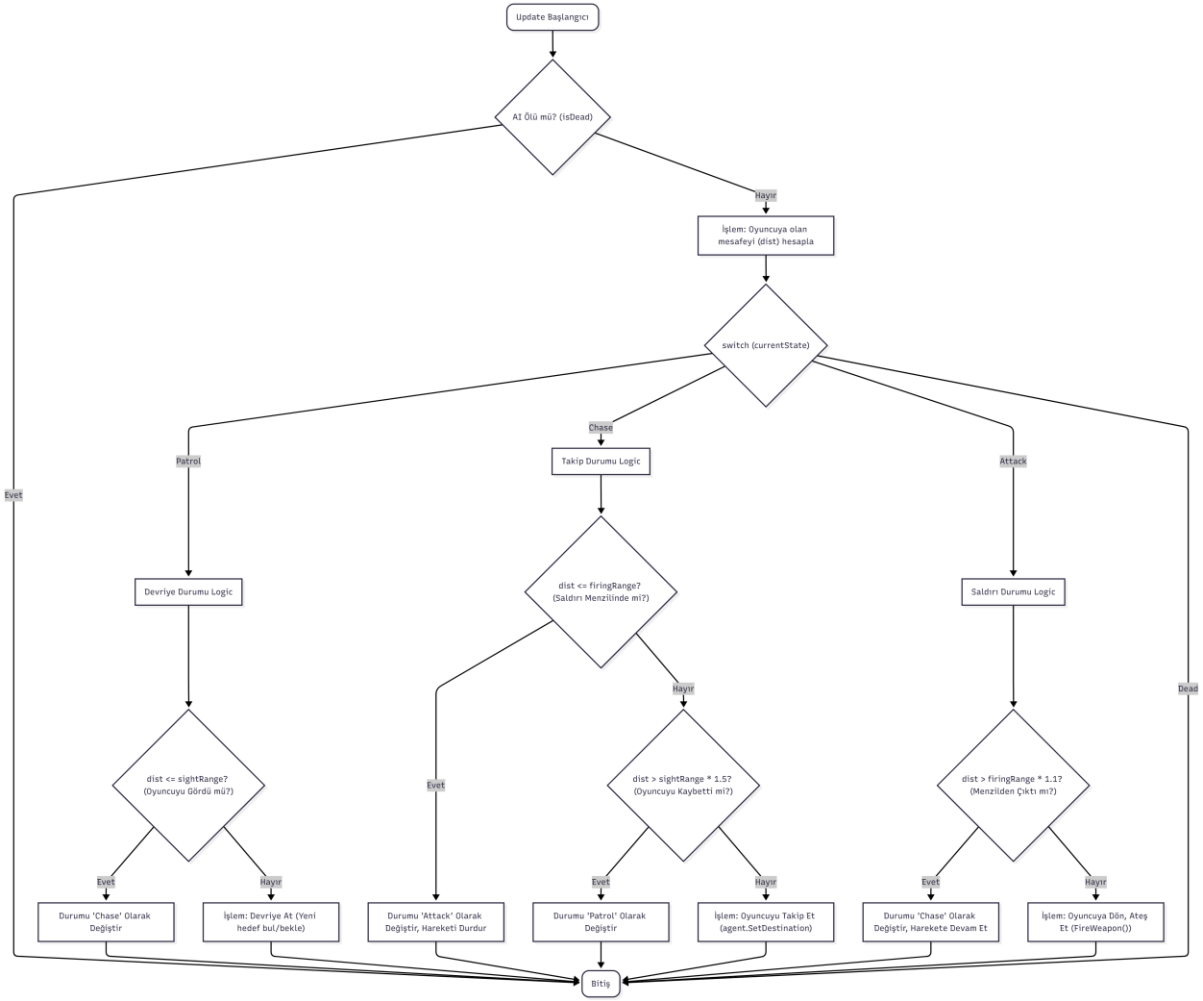
Şarjör değişirme mekaniği oyunun akışını kilitlememek için eş zamansız (asynchronous) bir Coroutine fonksiyonu (ReloadCoroutine) ile çözülmüştür. Yukarıdaki diyagramda gösterildiği gibi süreç başladığında mermiDolduruyor değişkeni 'true' olarak ayarlanır ve arayüzde 'Dolduruluyor...' yazar. Sistem kodda belirtilen 'reloadSuresi' kadar bekler. Bekleme tamamlandığında ihtiyaç duyulan mermi sayısı hesaplanır ve 'toplamMermi' envanteri kontrol edilir. Gerekli mermiler şarjöre eklenir, envanterden düşülür ve mermiDolduruyor değişkeni 'false' yapılarak arayüz son mermi durumuyla güncellenir. Bu sayede, uzun süreli bir işlem oyunun akışını kesintiye uğratmadan halledilmiştir.

Oyuncu Can ve Hasar Alma Sistemi



Oyuncunun can sistemi PlayerHealth.cs script'i tarafından yönetilir. Dışarıdan gelen hasarlar 'currentHealth' değişkenine doğrudan müdahale edemez bunun yerine TakeDamage() fonksiyonunu çağırmak zorundadır. Bu fonksiyon oyuncunun canını azaltır ve arayüzdeki 'HealthSlider'ı günceller. Eğer can sıfıra düşerse 'Die()' fonksiyonu tetiklenir. Ölüm sürecinde oyuncunun 'isDead' durumu 'true' olarak ayarlanır, movementScript ve shootingScript devre dışı bırakılarak kontrol engellenir, fiziksel kaymayı önlemek için Rigidbody dondurulur ve 'Die' animasyonu oynatılır. Son olarak 'ShowPanelRoutine' Coroutine i başlatılarak oyun sonu paneli gecikmeli olarak ekrana getirilir.

Yapay Zeka Durum Makinesi (State Machine)



Düşman yapay zekası bir Durum Makinesi (State Machine) tasarım deseni kullanılarak modellenmiştir. AI_Controller.cs script'i, Update() fonksiyonu içinde currentState adında bir değişkeni sürekli kontrol eder. Yapay zeka bu duruma göre 'Patrol' (devriye), 'Chase' (takip) veya 'Attack' (Saldırı) mantıklarından birini çalıştırır. Oyuncuya olan mesafeye bağlı olarak durumlar arasında geçiş yapılır. Örneğin 'Patrol' durumundayken oyuncu 'sightRange' (görüş mesafesi) içine girerse durum 'Chase' (takip) olarak değişir. Oyuncu 'firingRange' (ateş menzili) içine girerse durum 'Attack' (saldırı) olarak güncellenir ve 'FireWeapon()' fonksiyonu tetiklenir. Bu mimari karmaşık AI davranışlarının yönetilmesini ve genişletilmesini kolaylaştırmada işimize yaramıştır.

KULLANILAN YAZILIMSAL MİMARİLER VE TEKNİKLER

Nesne Yönelimli Programlama (OOP) Prensipleri

Proje geliştirilirken kodun sürdürülebilir, modüler ve genişletilebilir olması amacıyla Nesne Yönelimli Programlama (OOP) prensiplerine bağlı kalınmıştır. Özellikle

Kapsülleme ve Sorumlulukların ayrılması prensipleri kod kalitesini artırmak için aktif olarak kullanılmıştır.

Kapsülleme (Encapsulation) - PlayerHealth Örneği

Kapsülleme bir nesnenin iç verilerinin korunması ve bu verilere sadece kontrollü metotlar aracılığıyla erişilmesidir. PlayerHealth.cs script'imiz bu prensibe mükemmel bir örnektir. Oyuncunun can değeri olan currentHealth değişkeni 'private' olarak tanımlanmıştır. Düşman yapay zekası veya diğer sistemler oyuncunun canına doğrudan müdahale edemez. Bunun yerine 'public' olarak tanımlanmış TakeDamage(float amount) fonksiyonunu çağırmak zorundadırlar.

Sorumlulukların Ayrılması - Script'lerin (AtesEtme, PlayerHealth) ayrı olması)

Bu prensip her script'in tek bir sorumluluğu olması gerektiğini savunur. Projemizde tüm oyuncu fonksiyonlarını tek ve devasa bir 'Player' script'ine yazmak yerine sorumluluklar ayrılmıştır. Örneğin AtesEtme.cs sadece silah ve mermi mantığından sorumludur. NisanAlmaSistemi.cs sadece kamera geçişlerini yönetir.

Tasarım Desenleri: Durum Makinesi (State Machine)

Tasarım Desenleri sık karşılaşılan yazılım problemlerine getirilen kanıtlanmış çözümlerdir. Projemizdeki AI_Controller.cs script'i karmaşık yapay zeka davranışlarını yönetmek için 'Durum Makinesi' (State Machine) tasarım desenini kullanmaktadır. Bu desen yapay zekanın anlık olarak tek bir durumda (currentState) bulunmasını sağlar. Script'imiz AIState 'enum' yapısını kullanarak 'Patrol' (devriye), 'Chase' (takip) ve 'Attack' (saldırı) olmak üzere üç ana durum tanımlar.

Eşzamansız Programlama (Asynchronous): Coroutine'ler

Unity'de bazı işlemlerin oyunun ana döngüsünü kilitlemeden yapılması gerekir. Bu sorunu çözmek için 'Coroutine' adı verilen eşzamansız programlama tekniği kullanılmıştır. Projemizde iki kritik yerde bu teknikten faydalanılmıştır:

ReloadCoroutine() (AtesEtme.cs): Oyuncu şarjör değiştirdiğinde yield return new WaitForSeconds(reloadSuresi) komutu ile şarjörün dolması için gereken süre (örn: 2 saniye) kadar beklenir.

ShowPanelRoutine() (PlayerHealth.cs): Oyuncu öldüğünde 'Oyun Bitti' panelinin hemen ekrana gelmesi istenmez. Bu Coroutine sayesinde ölüm animasyonunun bitmesi için panelDelay (örn: 4 saniye) kadar beklenir.

Unity'e Özgü Teknikler

Proje geliştirme sürecinde sıfırdan kodlama yükünü ciddi ölçüde azaltan ve motorun kendi potansiyelini kullanan Unitye özgü güçlü teknolojilerden faydalanılmıştır. Bu paket ve sistemler projenin ana mekaniklerine daha fazla odaklanmamızı sağlamıştır.

Cinemachine Kamera Yönetimi: Projemizdeki üçüncü şahıs nişan alma (TPS) kamera sistemi manuel olarak Transform veya Quaternion hesaplamaları yapmak yerine Unity'nin 'Cinemachine' paketi ile çözülmüştür. NisanAlmaSistemi.cs script'i bu sistemin 'state-driven' yapısını kullanır. Script oyuncu nişan aldığında 'Aim Camera' objesinin 'Priority' (Öncelik) değerini 11'e, normal duruşta ise 'Hip Camera'nın önceliğini 9'a ayarlar. Unity'nin CinemachineBrain bileşeni bu öncelik değişimini algılayarak iki kamera arasındaki akıcı geçişi otomatik olarak yönetir. Bu yaklaşım karmaşık kamera geçiş mantığını sadece birkaç satır kodla çözmemizi sağlamıştır.

Animation Rigging (IK): Karakterlerin animasyon sırasında silahlarını gerçekçi bir şekilde tutması ve nişan alması, 'Animation Rigging' paketi kullanılarak sağlanmıştır. RigBuilder bileşeni karakterin ana animasyonları oynatılırken, 'IK' (Inverse Kinematics) rig'lerinin bu animasyonlara ek olarak çalışmasını sağlar. Örneğin AI_Controller.cs script'i yapay zeka 'Chase' veya 'Attack' durumuna geçtiğinde leftHandIK.weight ve aimIK.weight değerlerini Mathf.Lerp kullanarak akıcı bir şekilde 0'dan 1'e çeker. Bu sayede yapay zeka yürürken aynı zamanda nişan alabilir. Oyuncu script'i (AteEtme.cs) de silahın idlePose ve aimPose arasındaki hareketinden bağımsız olarak sol elin silaha kilitlenmesi için leftHandIKTarget ı kullanır.

NavMesh Navigasyonu: Düşman yapay zekasının harita üzerinde akıllıca hareket etmesi ve engellerden kaçınması Unity'nin 'NavMesh' (Navigation Mesh) sistemi ile sağlanmıştır. Editörde harita geometrisi 'NavMesh Surface' bileşeni kullanılarak 'bake' edilmiş ve yürünebilir alanlar hesaplanmıştır. AI_Controller script'i bu sistemin NavMeshAgent bileşenine komutlar gönderir. 'Chase' durumunda agent.SetDestination(playerTransform.position) komutu ile oyuncuyu otomatik olarak

takip eder. 'Patrol' durumunda ise NavMesh.SamplePosition kullanarak rastgele geçerli bir hedef nokta bulur ve oraya yönelir. Bu teknik karmaşık algoritmaları sıfırdan yazma ihtiyacını ortadan kaldırmıştır.

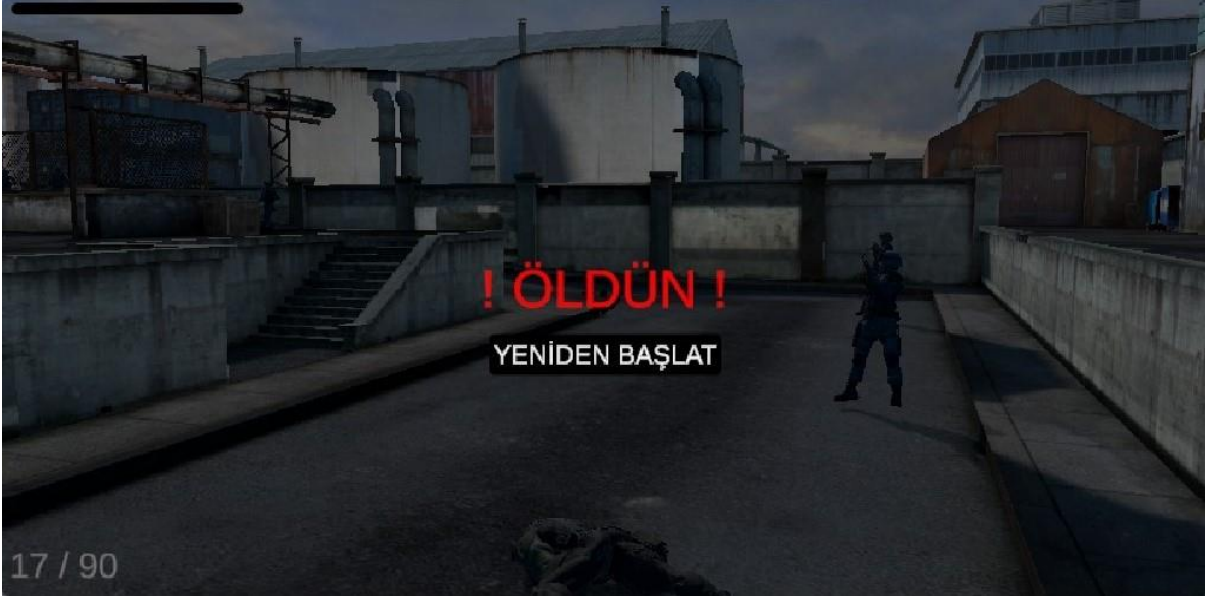
TASARLANAN ARAYÜZLER (UI/UX)

Oyun İçi Arayüz (HUD)



Oyun içi arayüz oyuncuya ihtiyaç duyduğu kritik bilgileri ekranı kalabalıklaştırmadan sunmak amacıyla tasarlanmıştır. Sol üst köşede HealthSlider bileşeni ile oyuncunun anlık can durumu görsel bir bar olarak gösterilmektedir. Sol alt köşede ise MermiYazisi bileşeni ile oyuncunun şarjöründeki (anlıkMermi) ve envanterindeki (toplamMermi) mermi miktarı gösterilir. Bu bilgiler PlayerHealth.cs ve AtesEtme.cs script'leri tarafından tetiklenen UpdateUI() fonksiyonları aracılığıyla anlık olarak güncellenmektedir.

Oyun Sonu Paneli (Game Over Panel)



Oyuncu öldüğünde yukarıdaki görselde bulunan 'Oyun Sonu Paneli' ekrana getirilir. Bu panelin oyuncunun ölüm animasyonunun bitmesi için panelDelay (örneğin: 4 saniye) gibi bir gecikmeyle gelmesi bir Coroutine ile sağlanmıştır. Panel aktif olduğunda oyunun kontrolü durdurulur ve mouse imleci görünür hale getirilir. Bu panel oyuncuya oyunu yeniden başlatma gibi seçenekler sunarak oyun döngüsünü tamamlar.

GRAFİK KULLANIMI VE OPTİMİZASYON

Kullanılan Grafik Varlıkları (Assets) ve Kaplamalar

Projenin görsel kalitesini oluşturmak için Unity Asset Store'dan temin edilen grafik varlıkları (assets) kullanılmıştır. Karakter modelimiz ve silah modelimiz oyunun bilim temasına uygun olarak seçilmiştir. Çevre tasarımı ve harita gerçekçi kaplamalar kullanılarak oluşturulmuştur.

Görsel Efektler (VFX) ve Işıklandırma

Oyun mekaniklerini görsel olarak desteklemek ve atmosferi güçlendirmek için Unity'nin 'Particle System' altyapısından faydalanılmıştır. Ateş etme mekanizmasını daha etkili hale getirmek için her atışta namluAtesiEfekti tetiklenmiştir. Benzer şekilde AI_Controller script'i de FireWeapon() fonksiyonu içinde hem kendi namlu ateşini hem de merminin çarptığı yerde bir 'mermi izi' efektini oluşturur. Bu görsel efektler oyuncunun aksiyonu daha net algılamasını sağlamıştır.

Optimizasyon Teknikleri

Oyunun akıcı bir performansla çalışması için temel optimizasyon teknikleri dikkate alınmıştır. Grafik varlıkları seçilirken yüksek performans sağlaması amacıyla 'low-poly' (düşük poligon sayılı) modeller tercih edilmiştir.

SÜREÇ YÖNETİMİ VE GITHUB KULLANIMI

Ekip Görev Dağılımı

Proje geliştirme süreci, proje üyeleri arasında net bir görev dağılımı yapılarak yürütülmüştür. Ekip içi koordinasyon kolayca sağlanmıştır. Görev dağılımı şu şekildedir:

Atakan ÇETLİ: Harita yüklenmesi ve düzenlenmesi, silah objesi, nişan alma sistemi, karakter hareket sistemi.

Muhammed Ali DERİNDAG: Silah envanter sistemi, ateş etme sistemi, oyuncu can sistemi.

Sadık GÜNAY: NPC, AI NavMesh navigator, animasyonlar, karakter ölme öldürme sistemi, oyun bitme ve başlama mekanikleri.

GitHub Versiyon Kontrol Süreci

Proje başından sonuna kadar bir Git deposu üzerinden yönetilmiş ve tüm kodlar/varlıklar GitHub üzerinde barındırılmıştır. Ekip olarak her üyenin projeye katkısının şeffaf bir şekilde izlenmesi için düzenli 'commit' işlemi uygulanmasına özen gösterilmiştir.

LİTERATÜR TARAMASI VE KARŞILAŞTIRMA

Örnek Çalışmaların İncelenmesi

Projemizin mekaniklerini geliştirmek için literatürde yer alan yaygın teknikler incelenmiştir. Özellikle yapay zeka (AI) ve üçüncü şahıs kamera (TPS) sistemleri için iki ana yaklaşım öne çıkmaktadır:

Yapay Zeka Mimarileri: Oyun geliştirmede yapay zeka davranışlarını modellemek için en yaygın iki yaklaşım projemizde kullandığımız 'Sonlu Durum Makineleri' ve 'Davranış Ağaçları'dır. Gelişmiş robotik ve 'F.E.A.R.' gibi büyük bütçeli video oyunları özellikle çok karmaşık ve modüler davranışlar (aynı anda hem siper alma hem ateş etme gibi)

gerektiğinde 'Davranış Ağaçları'nı tercih etmektedir. 'Davranış Ağaçları'nın 'Durum Makineleri'ne göre daha ölçeklenebilir ve çalışma zamanında değişikliğe daha esnek olduğu belirtilmektedir.

TPS Kamera Sistemleri: Üçüncü şahıs kamera sistemleri için literatürde iki ana yöntem bulunmaktadır. İlk Unity'nin kendi dökümantasyonlarında belirtildiği gibi 'Cinemachine' paketinin kullanılmasıdır. İkinci yöntem ise kameranın tüm hareketlerini, engellere çarpışma kontrollerini ve nişan alma geçişlerini manuel olarak kodlayan özel bir script yazmaktır.

Projemiz ile Literatürün Karşılaştırılması

Literatürdeki bu örneklerle göre projemizde yaptığımız teknik tercihler şu şekilde gerekçelendirilebilir:

Yapay Zeka Karşılaştırması: Projemizdeki AI_Controller script'i için literatürdeki daha karmaşık 'Davranış Ağaçları' yerine 'Durum Makinesi' (State Machine) deseni tercih edilmiştir. Bunun nedeni projemizin kapsamının 'Patrol', 'Chase' ve 'Attack' gibi basit, net ve belirli durumlarla sınırlı olmasıdır. 'Durum Makinesi' bu kapsamdaki bir yapay zeka için daha basit, anlaşılır, hızlı geliştirilebilir ve performanslı bir çözüm sunmuştur. 'Davranış Ağacı' mimarisinin sunduğu yüksek esneklik projemizin mevcut hedefleri için bir gereksinim olarak görülmemiştir.

Kamera Karşılaştırması: Kamera sistemi için literatürdeki 'manuel script' yazma yaklaşımı yerine modern ve profesyonel bir çözüm olan 'Cinemachine' paketi kullanılmıştır. NisanAlmaSistemi.cs script'imiz bu paketin 'Priority' (Öncelik) özelliğini kullanarak karmaşık kamera geçiş (blending) ve pozisyon hesaplama kodlarına ihtiyaç duymadan 'Hip' (normal) ve 'Aim' (nişan) kameraları arasında akıcı bir geçiş sağlamıştır. Bu tercih, geliştirme sürecini önemli ölçüde hızlandırmıştır.

KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜMLER

Teknik Zorluklar

Proje geliştirme sürecinde planlama aşamasında öngörülemeyen çeşitli teknik zorluklarla karşılaşmıştır.

Zorluk: En karmaşık zorluklardan biri oyuncu karakterinin nişan alma (aimPose) pozisyonuna geçtiğinde elinin AK47 silahını tam olarak doğru tutmamasıydı. 'Animation Rigging' (IK) sisteminde, leftHandIKTarget objesinin pozisyonu animasyonla birlikte kayıyordu.

Çözüm: Bu sorunu çözmek için, IK_Rig bileşenindeki 'weight' (ağırlık) değerleri ve leftHandIKTarget objesinin pozisyonu, animasyon döngüsünün farklı aşamalarında güncellenerek ve saatlerce ince ayar yapılarak, elin silaha kilitlenmesi sağlandı.

Zorluk: AI_Controller script'ini ilk test ettiğimizde yapay zeka karakterlerinin 'Chase' durumunda haritadaki dar koridorlarda veya engellerin köşelerinde takıldığı gözlemlendi.

Çözüm: Sorunun Unity'nin 'NavMesh' ayarlarından kaynaklandığı tespit edildi. Haritanın bileşeni daha düşük bir 'Agent Radius' değeriyle yeniden revize edildi. Ayrıca AI_Controller script'ine hedefe belirli bir mesafeden daha fazla yaklaşamazsa yeni bir hedef arama mantığı eklendi.

Süreç Yönetimi Zorlukları

Teknik zorlukların yanı sıra, bir ekip olarak eşzamanlı çalışmanın getirdiği süreç yönetimi zorlukları da yaşanmıştır.

Zorluk (GitHub Merge Çakışması): Ekip olarak en sık karşılaştığımız sorun, 'GitHub Merge Conflict' (Birleştirme Çakışması) olmuştur. Özellikle birden fazla üyenin aynı anda AtesEtme.cs script'i gibi merkezi bir script üzerinde veya aynı 'Scene' (Sahne) dosyası üzerinde değişiklik yapmaya çalışması bu çakışmalara yol açmıştır.

Çözüm: Bu sorunu aşmak için görev dağılımını daha net hale getirdik. Her üyenin projenin farklı script'leri üzerinde çalışmasını sağladık.

SONUÇ VE PROJENİN KATKILARI

Bu proje Yazılım Geliştirme Laboratuvarı dersi kapsamında Unity oyun motoru kullanılarak fonksiyonel bir üçüncü şahıs nişancı (TPS) oyunu geliştirme hedefiyle başlatılmıştır. Proje sonucunda hedeflenen ana mekaniklerin (nişan alma, ateş etme, düşman yapay zekası ve can sistemi) tamamı başarıyla uygulanmıştır.

Geliştirilen sistemler 'Durum Makinesi' mimarisi 'Nesne Yönelimli Programlama' prensipleri ve 'Cinemachine' gibi Unity teknolojileri kullanılarak hayata geçirilmiştir. Profesyonel çözümler incelenmiş ve projenin kapsamına en uygun teknikler seçilmiştir.

Bu projenin tamamlanması ekibimize hem teknik hem de yönetsel açıdan önemli katkılar sağlamıştır:

Teknik Kazanımlar: Bu proje sayesinde karmaşık 'Yapay Zeka' davranışlarının eş zamansız işlemlerin oyun akışını bozmadan nasıl yapılacağını ve 'Animation Rigging' ile

'Cinemachine' gibi profesyonel Unity paketlerinin bir projeye nasıl entegre edileceğini deneyimleyerek öğrendik.

Süreç Yönetimi Kazanımları: Teknik kazanımlardan daha da önemlisi bir ekip olarak ortak bir hedef için nasıl çalışılacağını deneyimledik. GitHub kullanarak versiyon kontrolü yapmanın, özellikle 'merge conflict' (kod çakışması) gibi sorunlarla karşılaştığımızda ekip içi iletişimin ve net görev dağılımının ne kadar kritik olduğunu anladık.

KAYNAKÇA

- https://assetstore.unity.com/packages/3d/environments/industrial/rpg-fps-game-assets-for-pc-mobile-industrial-set-v3-0-101429?srltid=AfmBOoqfyOPgHNAjW_CVaUD-BQRs1AEcLWjxJVDSelE0s_VN3yT5PXFfr
- <https://assetstore.unity.com/packages/3d/characters/humanoids/sci-fi/sci-fi-character-08-renegade-208640?srltid=AfmBOoo-e8WY6ET7xSGF9I7lQWZuh3XHCoPbAr3niygb8RrMTh4QP3MO>
- <https://assetstore.unity.com/packages/3d/props/weapons/3d-modern-weapons-guns-lowpoly-pack-lite-276794?srltid=AfmBOor7VfzlZUKKtm3J8lwd9uNPqBJMCDplSDlBAOxSHGXtdzoXxbwR>
- <https://assetstore.unity.com/packages/2d/textures-materials/sprite-muzzle-flashes-83068?srltid=AfmBOoqefz-f4Yu1ggv2ojTq94UNl5hhGmplP0AqZpjFAKv46ujWYzjQ>
- Google.com